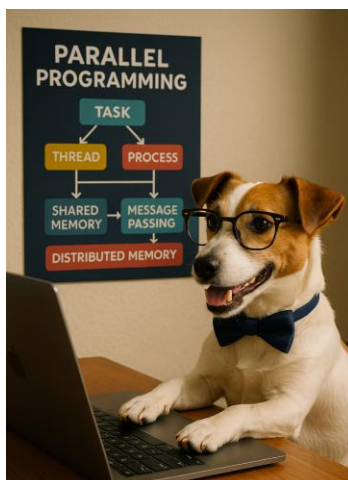




LAB 9, SPRING 2026

DATA PARALLELISM

LAB ASSIGNMENT DIRECTIONS



THIS LAB IS DIFFERENT FROM MOST THE OTHERS. WHY?

*The primary goal & challenge of this lab is not in learning to write code but in learning the advantages and basics of parallel programming. Modern software is built to run fast and efficiently on multi-core systems, and that means parallel programming is **no longer optional—it's essential**. Learning it early **gives you a serious edge**, making you more competitive, job-ready, and prepared to solve real-world problems at scale.*

REQUIRED SKILLS & KNOWLEDGE

1. C++ Vectors library
2. Parallel Programming / Data parallelism with OpenMP

ASSIGNMENT OBJECTIVE

- You will be able to write a C++ program that processes data in parallel.
- You will practice working with OpenMP to parallelize your program.

DESCRIPTION

You are given a complete program written with sequential instructions (**sequential.cpp**). Remember that sequential programming code is what we have been doing all semester – it is code that runs one step at a time, in the exact order it's written – from top to bottom. The given code does the following:

- creates a vector,
- fills the vector elements with strings read in from a text file,

- changes the vector elements,
- sorts the vector elements alphabetically in ascending order, and
- calculates the number of seconds (runtime) the program takes to process and sort the vector elements.

You are given multiple text files that all have a different number of strings you will use for comparison.

You are also given an **incomplete** source file that you will finish (**parallel.cpp**).

Your job in this assignment is to **finish writing** a C++ program (**parallel.cpp**) that performs the same activities with parallel regions instead of sequential to speed up the runtime. Then you will run both programs, record the execution run times of each, and then **document which version ran faster on your machine using the directions under “TESTING & REPORTING RESULTS”**.

PAIRED PROGRAMMING OPTION



You may complete this lab assignment alone or you have an OPTION to complete this lab with a lab partner using paired programming techniques discussed in previous labs. Make sure each partner uploads the same exact zip file to your **Lab 9** assignment in ilearn. **Each source file should have both of your names in the comment block at the top.** Both students will receive the same feedback and grade.

WARNING ABOUT ACADEMIC MISCONDUCT

Programming assignments in this course are your exams and should be only YOUR work.

- **You are not allowed to get help from friends, family, or other students (unless they are your partner in lab).**
- **You are also not allowed to use generative AI such as ChatGPT, Codex, or CoPilot to assist in writing your code.**

HOW TO COMPILE

The biggest challenge with this assignment is COMPILING YOUR CODE. Please read this section carefully and follow the instructions step-by-step. The sequential.cpp given file can be compiled normally like how you have done all semester. However, you will need to follow the directions below to compile parallel.cpp.

WINDOWS

If you are using the Windows operating system and the GCC compiler (like MinGW), then it already supports OpenMP by default. You only need to include **-fopenmp** when compiling and include the correct libraries.

COMPILING C++ WITH OPENMP

```
g++ -Wall -fopenmp parallel.cpp -o lab8
```

RUN YOUR PROGRAM

```
lab8
```

MAC

Using OpenMP on a Mac requires a bit of setup because Apple's default clang compiler does not support OpenMP by default. You'll need to install a version of the compiler that does support OpenMP, like GCC via Homebrew.

IS HOMEBREW ALREADY INSTALLED?

Open Terminal and run

```
which -s brew
```

If Homebrew is installed, you will see the full path to the brew executable, usually:

```
/opt/homebrew/bin/brew (if on Apple Silicon Macs) or  
/usr/local/bin/brew (if on Intel Macs)
```

If Homebrew is not installed, you will see:

```
brew not found
```

INSTALL HOMEBREW (IF IT ISN'T ALREADY INSTALLED)

In the Terminal, run (all on one line):

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

INSTALL GCC (WHICH SUPPORTS OPENMP)

```
brew install gcc
```

This installs a GCC version like gcc-14 or later, depending on the current release.

CHECK THE GCC VERSION

```
ls /opt/homebrew/bin | grep "g++" (if on Apple Silicon Macs) or  
ls /usr/local/bin | grep "g++" (if on Intel Macs)
```

COMPILING C++ WITH OPENMP

If your Homebrew installed GCC 14, the command will be:

```
g++-14 -fopenmp -Wall parallel.cpp -o lab8
```

Replace g++-14 with the version Homebrew installed (gcc-13, gcc-15, etc.)

RUN YOUR PROGRAM

```
./lab8
```

PROGRAM SPECIFICATIONS

STEP ONE – DOWNLOAD GIVEN FILES

Download **sequential.cpp**, **parallel.cpp**, and five text files listed below from the lab 9 assignment in ilearn and save it in your **Lab 9** folder.

1. **words_374.txt** – contains 374 strings (use this file to test if your code is sorting the strings but then comment out the function call statement to printVector for any of the other text files)
2. **words_9330.txt** – contains 9,330 strings
3. **words_18660.txt** – contains 18,660 strings
4. **words_31100.txt** – contains 31,100 strings
5. **words_466493.txt** – contains 466,493 strings

STEP TWO - REVIEW GIVEN CODE

Review the **sequential.cpp** program to see what is happening in this program. This program does the following:

1. Reads all the strings from the specified text file (name of text file acquired from the user).
2. Starts the timer (needed to calculate runtime of the for loop).
3. Uses a **for loop** to iterate through the string vector and does the following for each element in the vector.
 - a. Uses a for loop to count the number of characters in the string.
 - b. Converts the number of characters to a string.

- c. Prepends the number of characters to the front of the string.
 - d. Example: if the string is “apricot” then the resulting string after running this code would be “7_apricot” and “cat” would be changed to “3_cat”
4. Calls the **bubble sort** function, which will sort all the strings in alphabetical ascending order.
 - a. Bubble Sort is one of the simplest sorting algorithms, but it is also the slowest. This is fine if you only have a few elements to sort, but as the list of elements gets bigger, the time to sort the list grows significantly.
 - b. To learn more about the Bubble Sort algorithm, check out this article: <https://www.geeksforgeeks.org/bubble-sort-algorithm/>
 - c. Also, here is a 52-second visualization: <https://www.youtube.com/shorts/y2AghjB4Wxs>
5. Stops the timer.
6. Calculates the runtime.
7. Print the vector if you want to see the printed vector (only do this for smallest text file for testing the code and then comment it out for the larger text files).
8. Prints the computation time to the screen.

STEP THREE – MODIFY PARALLEL.CPP

Now, open **parallel.cpp** and add/modify code.

1. Update the comment block at the top of this source file with your name and today’s date.
2. Add the required #includes so that you can use OpenMP in this program.

Open the OpenMP slides that were also uploaded with this assignment to review what it is!

a. #include <vector>

This library is required to create vectors, use vectors, and call vector functions in your program. Vector is not a built-in data structure like an array and therefore, requires an additional library.

b. #include <algorithm>

This assignment uses a function from the algorithm library: **for_each()**

https://en.cppreference.com/w/cpp/algorithm/for_each.

c. #include <omp.h>

This library is required to use OpenMP, which is a way we can perform parallel computation for C, C++, and Fortran languages. More information on this library can be found on the OpenMP website at

<https://www.openmp.org/>

3. Read through the comments in the file and add the necessary code.
 - a. Explicitly set the number of **threads**. A thread is a logical branch of the program which can be executed in parallel with other threads of the same program. You will add the code below to set the number of threads to **4**. If you use too many threads (try 500), then you will see a slower runtime of the program due to the overhead of separating the tasks into threads and then combining the solution back together. If you put too few threads (try 2), then you will not see much speedup at all. **The ideal number of threads varies depending on your computer hardware and operating system.** The best way to figure it out is to test different numbers.

```
const int threads_wanted = 4;
omp_set_dynamic(false);
omp_set_num_threads(threads_wanted);
```

- b. Change the **for_each** loop to a traditional built-in C++ **for** loop and add your OpenMP directive before the loop begins. OpenMP will not work on the `for_each` function from the algorithm library, but works perfectly on the built-in `for` loop. Review the OpenMP slides to see what directive you should add in before a `for` loop. You will need to make sure to define count inside the loop so that it is a local variable to each separate thread running an iteration of this loop when it is running in parallel.
- c. Your output should look like the `sequential.cpp` output but should print "Computation time for parallel = X.XXX seconds." Where X.XXX is replaced by the actual computation time in seconds.

STEP FOUR – TESTING & REPORTING RESULTS

Once your program is compiling and working as expected, create a text file named **results.txt** and answer the following questions in this text file:

1. How many cores/processors are on your computer?
 - a. **Windows:** Press Ctrl + Shift + Esc to open Task Manager, then select the Performance tab. You can also press Ctrl+Alt+Delete, select Task Manager, click More details, then click Performance and CPU. Look for the number of cores on this screen.
 - b. **Mac:** Click the Apple icon, select About This Mac, then select System Report. In the System Information window, look for the Total Number of Cores.
2. Run **sequential.cpp** with each of the provided text files and each time, record the computation time in `results.txt`. NOTE: make sure the `printVector` function is not being called when recording computation time!
 - a. `words_374.txt`
 - b. `words_9330.txt`
 - c. `words_18660.txt`
 - d. `words_31100.txt`
 - e. `words_466493.txt` – **you do not have to complete running this text file sequentially – it takes hours – so only if you want to for fun**
3. Run **parallel.cpp** with each of the provided text files and each time, record the computation time in `results.txt`. NOTE: make sure the `printVector` function is not being called when recording computation time!
 - a. `words_374.txt`
 - b. `words_9330.txt`
 - c. `words_18660.txt`
 - d. `words_31100.txt`
 - e. `words_466493.txt`
4. Now, change the number of threads to **500** in **parallel.cpp** by changing the 4 to 500 for the **threads_wanted** variable. Then, compile & run the program again with **words_18660.txt**. What was the computation time?
5. Now, change the number of threads to **2** in **parallel.cpp** by changing the 500 to 2 for the **threads_wanted** variable. Then, compile & run the program again with `words_18660.txt`. What was the computation time?
6. **What was surprising about your results?** Was there anything that happened that you didn't expect?
7. **The sample output was created with a computer that has 8 cores. Did you get similar running times as the sample output?** Why or why not?

FILL OUT THE LAB REPORT

To get credit, you must upload a screenshot of the confirmation page to this lab assignment named **lab9ReportProof**.

Lab Report Link: https://tntech.co1.qualtrics.com/jfe/form/SV_6M4wwN1g5Ynv01w

WHAT TO TURN IN

Create a zip file named **YOUR-TTU-USERNAME_lab9** containing the following .cpp files and upload it to ilearn. Replace YOUR-TTU-USERNAME with your TTU username. Example: **acrockett43_lab9.zip**

- **sequential.cpp**
- **parallel.cpp**
- **all the .txt files that you were given**
- **results.txt**
- **Lab9ReportProof**

SAMPLE OUTPUT

SEQUENTIAL.CPP

Note: You will most likely not get the same compilation time as we did in this sample output because runtime is computer-dependent and is based on many factors. When running your executable files, ensure **all** your other software is closed to get the best running time possible for both programs. Turn off notifications as well. You may also want to pause your antivirus and disconnect from wifi.

Also, I cut off the output where the names of the text files are printed.

FILENAME: **words_374.txt**

```
*****  
Computation time for sequential = 0.00297 seconds.
```

FILENAME: **words_9330.txt**

```
*****  
Computation time for sequential = 3.18796 seconds.
```

FILENAME: **words_18660.txt**

```
*****  
Computation time for sequential = 10.95165 seconds.
```

FILENAME: **words_31100.txt**

```
*****  
Computation time for sequential = 41.49362 seconds.
```

FILENAME: **words_466493.txt**

```
*****  
Computation time for sequential = 19229.52540 seconds.
```

PARALLEL.CPP

Note: this is the computation time we got when running the program with 4 threads on a computer with 8 cores.

FILENAME: words_374.txt

```
*****  
Computation time for OpenMP parallel = .04293 seconds.
```

Parallel version is **slower** when number of elements is small due to overhead of thread creation & management.

FILENAME: words_9330.txt

```
*****  
Computation time for OpenMP parallel = 3.41816 seconds.
```

FILENAME: words_18660.txt

```
*****  
Computation time for OpenMP parallel = 8.25634 seconds.
```

I started to see parallel version run **faster** with 18,660 elements in the vector.

FILENAME: words_31100.txt

```
*****  
Computation time for OpenMP parallel = 21.62495 seconds.
```

PARALLEL.CPP WITH WORDS_466493.TXT

Enter the name of the text file: words_466493.txt

```
*****  
Computation time for OpenMP parallel = 5971.83695 seconds.
```